

Code-Layout, Readability and Reusability

Date	1 July 2026
Team ID	—
Project Name	Emotion Detection & Learning Support Engine
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes /No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Used standard Python indentation (4 spaces) throughout the project.
2	Proper File Structure	Files and folders are logically organized	Yes	Project follows a modular structure with separate folders for models (BiLSTM/BERT), Gemini prompt logic, Streamlit UI, and logging utilities.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables such as <code>problem_text</code> , <code>emotion_label</code> , <code>confidence_scores</code> , and <code>guidance_response</code> clearly indicate their purpose.
4	Function / Method Names	Functions are descriptively named	Yes	Functions such as <code>predict_emotion_bilstm()</code> , <code>predict_emotion_bert()</code> , <code>enhance_with_keywords()</code> , and <code>generate_guidance()</code> use meaningful names.
5	Code Comments	Inline and block comments explain logic	Yes	Major sections such as model loading, keyword enhancement, and Gemini prompt construction are documented using comments.

S.No	Code Quality Parameter	Description	Followed (Yes /No/Partial)	Remarks
6	Modular Design	Code is split into reusable functions/modules	Yes	Logic is divided into reusable modules for emotion classification, keyword enhancement, guidance generation, and interaction logging.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Shared preprocessing and logging functions are reused across the BiLSTM and BERT pipelines to avoid duplication.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try-except blocks are used around model inference and Gemini API calls, with fallback messages shown in the UI.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	BiLSTM Emotion Classifier	Python (PyTorch)	Primary emotion prediction pipeline	High
2	BERT Emotion Classifier	Python (Hugging Face Transformers)	Dual-model comparison view	High
3	Keyword Enhancement Module	Python (rule-based logic)	Refines predictions from both models	High
4	Gemini Guidance Generator	Python (Gemini API, Prompt Engineering)	Generates tips, next steps, and encouragement	High
5	Interaction Logger	Python (CSV)	Logs inputs, predictions, and responses for analytics	Medium

Overall Code Quality Assessment:

S.No	Component / Module Name	Language / Technology
1	BiLSTM Emotion Classifier	Python (PyTorch)
2	BERT Emotion Classifier	Python (Hugging Face Transformers)
3	Keyword Enhancement Module	Python (rule-based logic)
4	Gemini Guidance Generator	Python (Gemini API)
5	Interaction Logger	Python (CSV)